

Année Universitaire 2025/2026

Simulateur du Problème des Lecteurs-Écrivains

Programmation Concurrente - Mini Projet

Membres du groupe :

EL Mazyani Mohamed Ilyas

Marouane mellakh

IDrissa abou nana-aichatou

Essadki Naoufal

Moussaly Chayma

15 mai 2026

Table des matières

1	Contexte et objectifs du travail	1
1.1	Contexte	1
1.2	Objectifs du travail	1
2	Structure du projet	1
2.1	Organisation des fichiers	1
2.2	Description des classes	2
3	Explication détaillée du code source	2
3.1	Classe Main - Point d'entrée	2
3.2	Classe RessourcePartagee - Le cœur de la synchronisation	3
3.3	Classe Simulateur - Orchestrateur	4
3.4	Classe Lecteur - Le thread lecteur	6
3.5	Classe Ecrivain - Le thread écrivain	7
3.6	Classe Log - Journalisation dans un fichier	7
3.7	Intégration de Log dans le projet	11
3.8	Exemple de fichier de log généré	11
4	Implémentation de l'interface graphique	12
4.1	Interface au démarrage	12
4.2	Déroulement d'une simulation	13
4.3	Fin de simulation et statistiques	14
5	Compilation et exécution	14
6	Conclusion	14
6.1	Résumé du travail réalisé	14
6.2	Difficultés rencontrées	15
6.3	Conclusion générale	15

1 Contexte et objectifs du travail

1.1 Contexte

Le problème des lecteurs-écrivains est un problème classique de la programmation concurrente. Il met en évidence les difficultés de synchronisation lorsqu'un ensemble de processus (lecteurs) souhaitent lire une ressource partagée tandis que d'autres (écrivains) souhaitent la modifier simultanément.

1.2 Objectifs du travail

Les objectifs fixés par le cahier des charges sont :

1. **Modélisation** : Implémenter des threads représentant les lecteurs et les écrivains.
2. **Synchronisation** : Créer des mécanismes permettant aux lecteurs de lire simultanément et aux écrivains d'écrire en exclusivité.
3. **Journalisation** : Enregistrer et afficher toutes les actions pour faciliter l'analyse.
4. **Visualisation** : Développer une interface graphique pour visualiser en temps réel les interactions.
5. **Prévention de la famine** : Implémenter une stratégie FIFO (premier arrivé, premier servi).

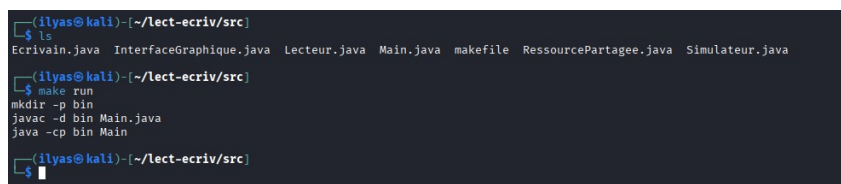
2 Structure du projet

2.1 Organisation des fichiers

Le projet est organisé de la manière suivante :

lect-ecriv/

Main.java	# Point d'entrée du programme
InterfaceGraphique.java	# Interface graphique (Swing)
Simulateur.java	# Orchestrateur de la simulation
RessourcePartagee.java	# Ressource partagée avec verrou FIFO
Lecteur.java	# Comportement d'un lecteur
Ecrivain.java	# Comportement d'un écrivain
Log.java	# Journalisation dans un fichier .log
makefile	# Script de compilation



```
(ilyas@kali)~/lect-ecriv/src
$ ls
Ecrivain.java  InterfaceGraphique.java  Lecteur.java  Main.java  makefile  RessourcePartagee.java  Simulateur.java
(ilyas@kali)~/lect-ecriv/src
$ make run
mkdir -p bin
javac -d bin Main.java
java -cp bin Main
(ilyas@kali)~/lect-ecriv/src
$
```

FIGURE 1 – Affichage de la structure des fichiers dans le terminal

Explication : Cette capture montre la liste des fichiers sources du projet. La commande `make run` permet de compiler et d'exécuter le programme.

2.2 Description des classes

Classe	Rôle
Main	Point d'entrée, lance l'interface graphique
InterfaceGraphique	Fenêtre Swing, contient les contrôles et l'affichage
Simulateur	Crée, démarre et arrête les threads
RessourcePartagee	Ressource protégée par un verrou FIFO
Lecteur	Thread qui lit la ressource
Ecrivain	Thread qui écrit dans la ressource
Log	Journalisation dans un fichier .log horodaté

3 Explication détaillée du code source

3.1 Classe Main - Point d'entrée

```
import javax.swing.SwingUtilities;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                InterfaceGraphique gui = new InterfaceGraphique();
                gui.setVisible(b: true);
            }
        });
    }
}
```

FIGURE 2 – Code de la classe Main

Explication ligne par ligne :

- `SwingUtilities.invokeLater()` : Cette méthode garantit que l'interface graphique est créée dans le thread dédié (Event Dispatch Thread). C'est une obligation en Swing pour éviter les problèmes de concurrence.
- `new InterfaceGraphique()` : Crée une nouvelle instance de la fenêtre principale.
- `setVisible(true)` : Rend la fenêtre visible à l'écran.

3.2 Classe RessourcePartagee - Le cœur de la synchronisation

```
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class RessourcePartagee {

    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock(fair: true);
    private String donnee = "Initial";
    private InterfaceGraphique gui;
    private static int compteurFile = 0;

    public RessourcePartagee(InterfaceGraphique gui) {
        this.gui = gui;
        gui.ajouterMessage("Ressource initialisee: " + donnee);
    }

    public void lire(int id) {
        int position;
        synchronized(this) {
            compteurFile++;
            position = compteurFile;
        }
        gui.ajouterMessage("[FILE] Lecteur " + id + " entre en file (position " + position + ")");

        rwLock.readLock().lock();
        gui.ajouterMessage("[LECTURE] Lecteur " + id + " lit: " + donnee);
        rwLock.readLock().unlock();

        gui.ajouterMessage("[SORTIE] Lecteur " + id + " sort de la file");

        try { Thread.sleep(100); } catch (Exception e) {}
    }

    public void ecrire(int id, String valeur) {
        int position;
        synchronized(this) {
            compteurFile++;
            position = compteurFile;
        }
        gui.ajouterMessage("[FILE] Ecrivain " + id + " entre en file (position " + position + ")");

        rwLock.writeLock().lock();
        gui.ajouterMessage("[ECRITURE] Ecrivain " + id + " ecrit: " + valeur);
        donnee = valeur;
        gui.majValeur(valeur);
        rwLock.writeLock().unlock();

        gui.ajouterMessage("[SORTIE] Ecrivain " + id + " sort de la file");

        try { Thread.sleep(300); } catch (Exception e) {}
    }

    public String getDonnee() {
        return donnee;
    }
}
```

FIGURE 3 – Code de la classe RessourcePartagee

Explication détaillée :

- `ReentrantReadWriteLock(true)` : Crée un verrou avec mode équitable. Le paramètre `true` garantit l'ordre FIFO (premier arrivé, premier servi). C'est cette ligne qui empêche la famine.
- `readLock().lock()` : Verrouillage en lecture. Plusieurs lecteurs peuvent prendre ce verrou simultanément. C'est ce qui permet aux lecteurs de lire en parallèle.
- `writeLock().lock()` : Verrouillage en écriture. Un seul thread peut prendre ce verrou à la fois. Pendant ce temps, aucun lecteur ne peut lire.
- `synchronized(this)` : Bloc synchronisé qui protège l'incrémentation du compteur de file. Cela garantit que chaque thread reçoit un numéro unique.
- Les messages `[FILE]`, `[LECTURE]`, `[ECRITURE]`, `[SORTIE]` permettent de visualiser l'ordre d'exécution dans l'interface.

3.3 Classe Simulateur - Orchestrateur

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class Simulateur {
5
6      private int nbLecteurs;
7      private int nbEcrivains;
8      private int dureeSimulationMs;
9
10     private RessourcePartagee ressource;
11     private List<Thread> threads;
12     private boolean fini;
13     private InterfaceGraphique gui;
14
15     public Simulateur(InterfaceGraphique gui, int nbLecteurs, int nbEcrivains, int dureeSimulationMs) {
16         this.gui = gui;
17         this.nbLecteurs = nbLecteurs;
18         this.nbEcrivains = nbEcrivains;
19         this.dureeSimulationMs = dureeSimulationMs;
20         this.ressource = new RessourcePartagee(gui);
21         this.threads = new ArrayList<>();
22         this.fini = false;
23     }
24
25     public void demarrer() {
26         for (int i = 1; i <= nbLecteurs; i++) {
27             Thread lecteur = new Thread(new Lecteur(i, ressource, this, gui));
28             lecteur.start();
29             threads.add(lecteur);
30             gui.ajouterMessage("Lecteur " + i + " démarre");
31             try { Thread.sleep(500); } catch (Exception e) {}
32         }
33
34         for (int i = 1; i <= nbEcrivains; i++) {
35             Thread ecrivain = new Thread(new Ecrivain(i, ressource, this, gui));
36             ecrivain.start();
37             threads.add(ecrivain);
38             gui.ajouterMessage("Ecrivain " + i + " démarre");
39             try { Thread.sleep(500); } catch (Exception e) {}
40         }
41
42         gui.ajouterMessage(message: "");
43         gui.ajouterMessage(">>> SIMULATION EN COURS (" + (dureeSimulationMs/1000) + " secondes) <<<");
44
45         try {
46             Thread.sleep(dureeSimulationMs);
47         } catch (Exception e) {}
48
49         arreter();
50     }
51
52     public void arreter() {
53         gui.ajouterMessage(message: "");
54         gui.ajouterMessage(">>> ARRÊT DE LA SIMULATION <<<");
55     }
56

```

FIGURE 4 – Code de la classe Simulateur - Démarrage

Explication du démarrage :

- Le constructeur initialise les paramètres : nombre de lecteurs, nombre d'écrivains, durée de la simulation.
- La méthode `demarrer()` crée et lance tous les threads :
 - `new Thread(new Lecteur(...))` : Crée un nouveau thread pour un lecteur.
 - `start()` : Démarre le thread (la méthode `run()` s'exécute alors en parallèle).
 - `threads.add()` : Stocke le thread dans une liste pour pouvoir l'arrêter plus tard.
 - `Thread.sleep(500)` : Pause de 500ms entre chaque démarrage pour mieux visualiser l'ordre d'arrivée.
- `Thread.sleep(dureeSimulationMs)` : Attend la durée configurée avant d'arrêter la simulation.

```
public void arreter() {
    gui.ajouterMessage(message: "");
    gui.ajouterMessage(message: ">>> ARRET DE LA SIMULATION <<<");

    fini = true;

    for (Thread t : threads) {
        t.interrupt();
    }

    for (Thread t : threads) {
        try {
            t.join(millis: 1000);
        } catch (Exception e) {}
    }

    gui.ajouterMessage(message: "");
    gui.ajouterMessage(message: "=== STATISTIQUES FINALES ===");
    gui.ajouterMessage("Lectures: " + Lecteur.getCompteurLectures());
    gui.ajouterMessage("Ecritures: " + Ecrivain.getCompteurEcritures());
    gui.ajouterMessage("Valeur finale: " + ressource.getDonnee());

    gui.majLectures(Lecteur.getCompteurLectures());
    gui.majEcritures(Ecrivain.getCompteurEcritures());
}

public boolean estFini() {
    return fini;
}
```

FIGURE 5 – Code de la classe Simulateur - Arrêt

Explication de l'arrêt :

- `fini = true` : Drapeau signalant la fin de la simulation. Les threads vérifient cette variable dans leur boucle.
- `t.interrupt()` : Demande poliment à chaque thread de s'arrêter. Cela réveille les threads qui sont en train de dormir.
- `t.join(1000)` : Attend que le thread se termine (maximum 1 seconde). Cela garantit que tous les threads sont bien arrêtés avant d'afficher les statistiques.
- `getCompteurLectures()` et `getCompteurEcritures()` : Récupèrent les statistiques finales.

3.4 Classe Lecteur - Le thread lecteur

```
import java.util.Random;

public class Lecteur implements Runnable {
    private int id;
    private RessourcePartagee ressource;
    private Simulateur simulateur;
    private InterfaceGraphique gui;
    private static int compteur = 0;

    public Lecteur(int id, RessourcePartagee ressource, Simulateur simulateur, InterfaceGraphique gui) {
        this.id = id;
        this.ressource = ressource;
        this.simulateur = simulateur;
        this.gui = gui;
    }

    public static synchronized int getCompteurLectures() {
        return compteur;
    }

    private static synchronized void incrementerCompteur() {
        compteur++;
    }

    public void run() {
        Random rand = new Random();

        while (!simulateur.estFini() && !Thread.currentThread().isInterrupted()) {
            try {
                Thread.sleep(1500 + rand.nextInt(bound: 2000));
            } catch (Exception e) {
                break;
            }

            ressource.lire(id);
            incrementerCompteur();
            gui.majLectures(compteur);
        }

        gui.ajouterMessage("Lecteur " + id + " arrete");
    }
}
```

FIGURE 6 – Code de la classe Lecteur

Explication détaillée :

- `implements Runnable` : Cette interface permet d'exécuter ce code dans un thread. La méthode `run()` contient le code à exécuter.
- `while (!simulateur.estFini() && !Thread.currentThread().isInterrupted())` : La boucle continue tant que la simulation n'est pas finie ET que le thread n'a pas reçu d'ordre d'arrêt.
- `Thread.sleep(1500 + rand.nextInt(2000))` : Pause aléatoire entre 1500ms et 3500ms. Cela simule le temps de réflexion entre deux lectures.
- `ressource.lire(id)` : Appelle la méthode de lecture synchronisée. C'est ici que le thread prend le verrou de lecture.
- `incrementerCompteur()` : Incrémente le compteur total de lectures (partagé entre tous les lecteurs).
- `synchronized` sur les méthodes statiques : Protège l'accès au compteur partagé. Cela évite que deux lecteurs modifient le compteur en même temps.

3.5 Classe Ecrivain - Le thread écrivain

```
public class Ecrivain implements Runnable {
    private int id;
    private RessourcePartagee ressource;
    private Simulateur simulateur;
    private InterfaceGraphique gui;
    private static int compteurEcritures = 0;
    private static int compteurModifs = 0;

    public Ecrivain(int id, RessourcePartagee ressource, Simulateur simulateur, InterfaceGraphique gui) {
        this.id = id;
        this.ressource = ressource;
        this.simulateur = simulateur;
        this.gui = gui;
    }

    public static synchronized int getCompteurEcritures() {
        return compteurEcritures;
    }

    private static synchronized String getNouvelleValeur(int id) {
        compteurModifs++;
        return "Donnee-" + compteurModifs + "-par-" + id;
    }

    private static synchronized void incrementerCompteurEcritures() {
        compteurEcritures++;
    }

    public void run() {
        Random rand = new Random();

        while (!simulateur.estFini() && !Thread.currentThread().isInterrupted()) {
            try {
                Thread.sleep(2000 + rand.nextInt(bound: 2500));
            } catch (Exception e) {
                break;
            }

            String valeur = getNouvelleValeur(id);
            ressource.ecrire(id, valeur);
            incrementerCompteurEcritures();
            gui.majEcritures(compteurEcritures);
        }

        gui.ajouterMessage("Ecrivain " + id + " arrete");
    }
}
```

FIGURE 7 – Code de la classe Ecrivain

Explication détaillée :

- Pause aléatoire plus longue : `2000 + rand.nextInt(2500)` (2000ms à 4500ms). Les écrivains sont moins fréquents que les lecteurs.
- `getNouvelleValeur()` : Méthode synchronisée qui :
 - Incrémente le compteur de modifications
 - Retourne une chaîne unique au format `Donnee-X-par-id`
- `ressource.ecrire(id, valeur)` : Appelle la méthode d'écriture synchronisée. C'est ici que le thread prend le verrou d'écriture (exclusif).
- `incrementerCompteurEcritures()` : Incrémente le compteur total d'écritures.

3.6 Classe Log - Journalisation dans un fichier

Pour conserver une trace complète de chaque simulation, nous avons implémenté une classe Log dédiée. Elle permet d'enregistrer toutes les actions (entrées dans la file, lectures, écritures, statistiques) dans un fichier texte horodaté.

```
J Logjava > ...
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Log {

    private static final DateTimeFormatter FMT_FICHIER =
        DateTimeFormatter.ofPattern(pattern: "yyyy-MM-dd_HH-mm-ss");
    private static final DateTimeFormatter FMT_HORODATAGE =
        DateTimeFormatter.ofPattern(pattern: "HH:mm:ss.SSS");
    private static final DateTimeFormatter FMT_LISIBLE =
        DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy HH:mm:ss");

    private static BufferedWriter writer = null;
    private static String fichier = null;

    /**
     * Initialise un nouveau fichier de log pour la simulation courante.
     * Crée l'en-tête avec la configuration de la simulation.
     * Doit être appelé une fois au démarrage, avant de créer le simulateur.
     *
     * @param nbLecteurs    nombre de lecteurs configurés
     * @param nbEcrivains   nombre d'écrivains configurés
     * @param dureeSecondes durée de la simulation en secondes
     */
    public static synchronized void initialiser(int nbLecteurs, int nbEcrivains, int dureeSecondes) {
        fermerSilencieux(); // ferme l'éventuel log précédent

        String horodatage = LocalDateTime.now().format(FMT_FICHIER);
        fichier = "simulation_" + horodatage + ".log";
    }
}
```

FIGURE 8 – Classe Log - Initialisation et attributs

Explication de la capture :

- `DateTimeFormatter` : Trois formats de dates différents sont définis :
 - `FMT_FICHIER` : Pour nommer le fichier (`yyyy-MM-dd_HH-mm-ss`)
 - `FMT_HORODATAGE` : Pour horodater chaque ligne (`HH:mm:ss.SSS`)
 - `FMT_LISIBLE` : Pour l'en-tête et le pied (`dd/MM/yyyy HH:mm:ss`)
- `writer` : Buffer d'écriture pour le fichier.
- `fichier` : Nom du fichier de log.
- `initialiser()` : Crée un nouveau fichier avec un nom unique basé sur la date et l'heure.

```

try {
    writer = new BufferedWriter(new FileWriter(fichier, append: false));
    ecrireLigne(ligne: "=====");
    ecrireLigne(ligne: "    SIMULATION LECTEURS-ECRIVAINS");
    ecrireLigne(ligne: "    Demarree le : " + LocalDateTime.now().format(FMT_LISIBLE));
    ecrireLigne(ligne: "=====");
    ecrireLigne(ligne: "    Lecteurs    : " + nbLecteurs);
    ecrireLigne(ligne: "    Ecrivains   : " + nbEcrivains);
    ecrireLigne(ligne: "    Duree      : " + dureeSecondes + " secondes");
    ecrireLigne(ligne: "=====");
    ecrireLigne(ligne: "");
} catch (IOException e) {
    System.err.println("[Log] Impossible de creer le fichier : " + fichier);
    System.err.println("[Log] Cause : " + e.getMessage());
    writer = null;
}

/**
 * Enregistre un message horodaté dans le fichier de log.
 * Thread-safe : peut être appelé depuis n'importe quel thread.
 * Les lignes vides sont transmises telles quelles (sans horodatage).
 *
 * @param message le texte à enregistrer
 */
public static synchronized void enregistrer(String message) {
    if (writer == null) return;

    if (message == null || message.trim().isEmpty()) {
        ecrireLigne(ligne: "");
    } else {
        String horodatage = LocalDateTime.now().format(FMT_HORODATAGE);
        ecrireLigne(ligne: "[" + horodatage + "] " + message);
    }
}

```

FIGURE 9 – Classe Log - Création du fichier et en-tête

Explication de la capture :

- Le fichier est créé avec `FileWriter` et `BufferedWriter`.
- L'en-tête contient :
 - La date et l'heure de début de simulation
 - Le nombre de lecteurs configurés
 - Le nombre d'écrivains configurés
 - La durée de la simulation
- En cas d'erreur (ex : permission refusée), un message d'erreur s'affiche dans la console.

```

/**
 * Enregistre les statistiques finales et ferme le fichier proprement.
 * Doit être appelé à la fin de chaque simulation, dans Simulateur.arreter().
 */
@param nbLectures  nombre total de lectures effectuées
@param nbEcritures nombre total d'écritures effectuées
@param valeurFinale valeur finale de la ressource partagée
*/
public static synchronized void fermerAvecStatistiques(
    int nbLectures, int nbEcritures, String valeurFinale) {

    if (writer == null) return;

    ecrireLigne(ligne: "");
    ecrireLigne(ligne: "=====");
    ecrireLigne(ligne: "  STATISTIQUES FINALES");
    ecrireLigne(ligne: "-----");
    ecrireLigne("  Lectures    : " + nbLectures);
    ecrireLigne("  Ecritures   : " + nbEcritures);
    ecrireLigne("  Valeur      : " + valeurFinale);
    ecrireLigne(ligne: "=====");
    ecrireLigne("  Terminee le : " + LocalDateTime.now().format(FMT_LISIBLE));
    ecrireLigne(ligne: "=====");

    String f = fichier;
    fermerSilencieux();
    System.out.println("[Log] Simulation enregistrée dans : " + f);
}

/**
 * Retourne le nom du fichier de log actuellement ouvert,
 * ou null si aucun fichier n'est ouvert.
 */
public static synchronized String getFichier() {
    return fichier;
}

```

FIGURE 10 – Classe Log - Enregistrement des messages et statistiques finales

Explication de la capture :

- `enregistrer(String message)` : Ajoute une ligne horodatée dans le fichier.
 - Format : `[+HH:mm:ss.SSS] message`
 - Les lignes vides sont écrites sans horodatage.
 - La méthode est `synchronized` pour être thread-safe.
- `fermerAvecStatistiques()` : Écrit les statistiques finales et ferme le fichier.
 - Affiche le nombre total de lectures et d'écritures.
 - Affiche la valeur finale de la ressource.
 - Affiche la date et l'heure de fin.
 - Un message dans la console indique le nom du fichier créé.

```

// -----
// Méthodes internes
// -----

/** Ferme le writer sans écrire de statistiques. */
private static void fermerSilencieux() {
    if (writer != null) {
        try {
            writer.flush();
            writer.close();
        } catch (IOException ignored) {}
        finally {
            writer = null;
        }
    }
}

/** Écrit une ligne brute dans le fichier (sans horodatage automatique). */
private static void ecrireLigne(String ligne) {
    try {
        writer.write(ligne);
        writer.newLine();
        writer.flush();
    } catch (IOException e) {
        System.err.println("[Log] Erreur d'écriture : " + e.getMessage());
    }
}
}

```

FIGURE 11 – Classe Log - Méthodes internes

Explication de la capture :

- `fermerSilencieux()` : Ferme le writer proprement sans écrire de statistiques. Les erreurs sont ignorées.
- `ecrireLigne(String ligne)` : Écrit une ligne brute dans le fichier (sans horodatage). Utilisée par `enregistrer()` et `fermerAvecStatistiques()`.

3.7 Intégration de Log dans le projet

Pour utiliser la journalisation dans le fichier, nous avons modifié `Simulateur.java` :

- `Log.initialiser(nbLecteurs, nbEcrivains, dureeSecondes)` est appelée au début de la simulation.
- Chaque action (lecture, écriture, entrée/sortie de file) est enregistrée avec `Log.enregistrer(mess`
- À la fin, `Log.fermerAvecStatistiques(lectures, ecritures, valeurFinale)` est appelée dans `arreter()`.

3.8 Exemple de fichier de log généré

Voici un exemple de fichier `simulation_2025-05-15_14-30-25.log` :

```

=====
SIMULATION LECTEURS-ECRIVAINS
Demarree le : 15/05/2025 14:30:25
-----

Lecteurs : 3
Ecrivains : 2

```

```

Duree : 5 secondes
=====

[+14:30:25.123] >>> SIMULATION EN COURS <<<
[+14:30:25.456] [DEMARRE] Lecteur 1
[+14:30:25.657] [FILE] Lecteur 1 entre en file (position 1)
[+14:30:25.658] [SORTIE] Lecteur 1 sort de la file et lit
[+14:30:25.659] [LECTURE] Lecteur 1 lit: Initial
[+14:30:25.760] [FIN] Lecteur 1 a termine
...
[+14:30:30.123] >>> ARRET DE LA SIMULATION <<<
[+14:30:30.125]
=====
STATISTIQUES FINALES
--
Lectures : 12
Ecritures : 5
Valeur : Donnee-5-par-2
=====
Terminee le : 15/05/2025 14:30:30
=====

```

4 Implémentation de l'interface graphique

4.1 Interface au démarrage

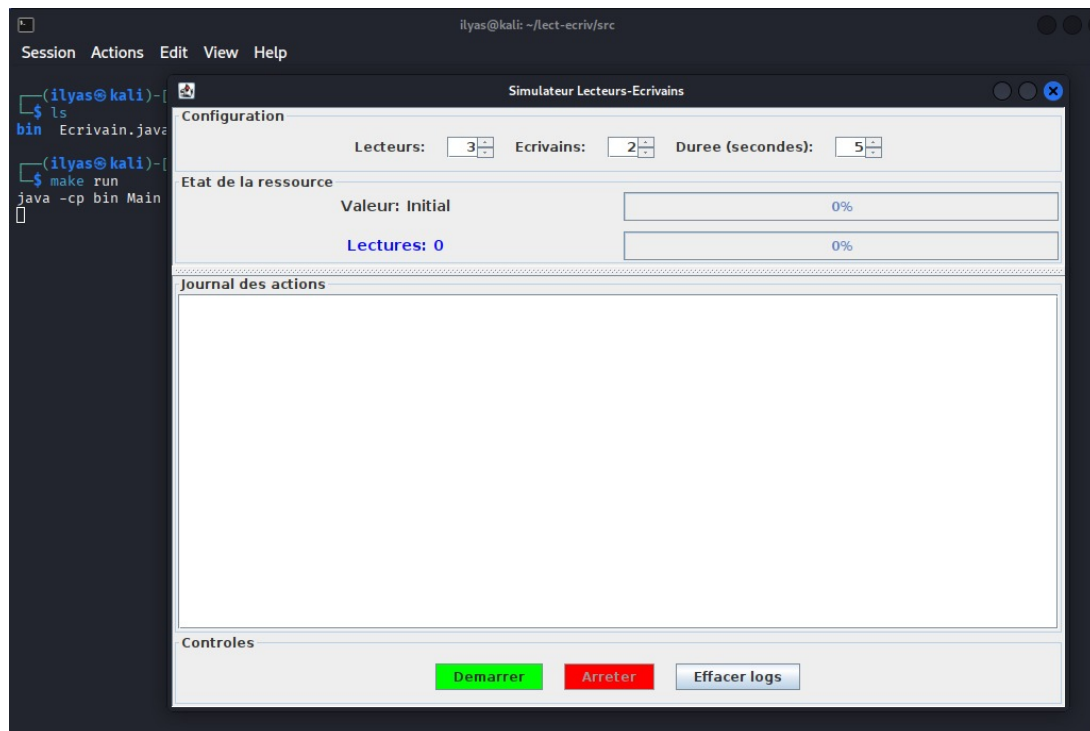


FIGURE 12 – Interface graphique au démarrage

Description des composants :

- **Zone Configuration** : Permet de modifier :
 - Le nombre de lecteurs (1 à 10)
 - Le nombre d'écrivains (1 à 10)
 - La durée de la simulation (1 à 30 secondes)
- **Zone Etat de la ressource** : Affiche en temps réel :
 - La valeur actuelle de la ressource
 - Le nombre total de lectures
 - Le nombre total d'écritures
- **Zone Journal des actions** : Affiche toutes les actions avec horodatage.
- **Boutons de contrôle** :
 - Démarrer : Lance la simulation
 - Arrêter : Interrompt la simulation
 - Effacer logs : Vide le journal

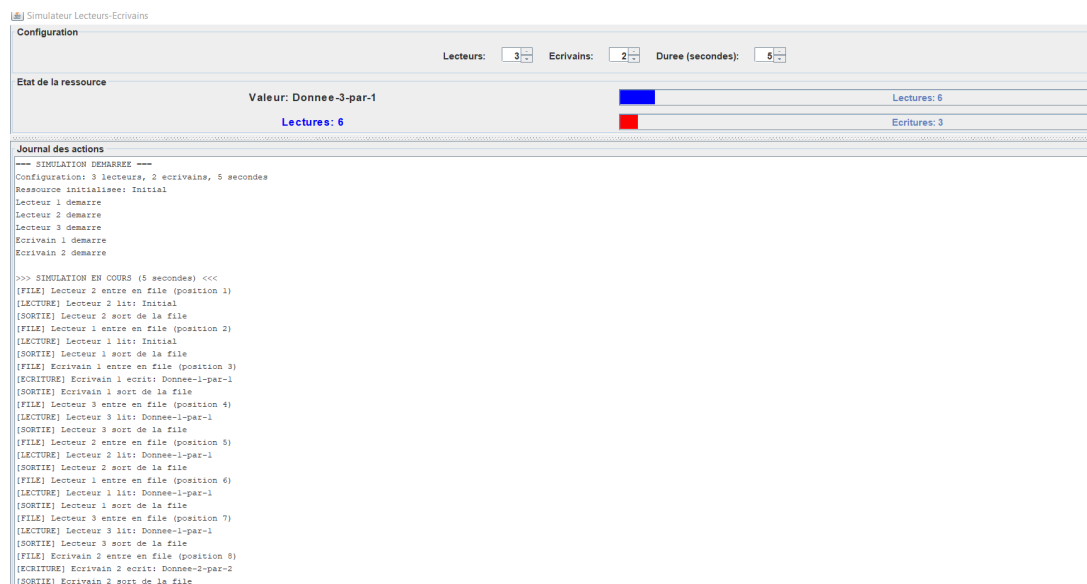
4.2 Déroulement d'une simulation

FIGURE 13 – Déroulement de la simulation - Journal des actions

Explication des messages :

- **[FILE]** Lecteur X entre en file (position N) : Le lecteur demande l'accès et prend sa place dans la file FIFO.
- **[LECTURE]** Lecteur X lit : Valeur : Le lecteur a obtenu le verrou et lit la ressource.
- **[SORTIE]** Lecteur X sort de la file : Le lecteur a terminé et libère le verrou.
- **[ECRITURE]** Ecrivain X écrit : Nouvelle valeur : L'écrivain modifie la ressource.

4.3 Fin de simulation et statistiques

```
[FILE] Ecrivain 1 entre en file (position 9)
[ECRIURE] Ecrivain 1 écrit: Donnee-3-par-1
[ROBITE] Ecrivain 1 sort de la file

>> ARRET DE LA SIMULATION <<
Ecrivain 1 arrete
Ecrivain 3 arrete
Lecteur 2 arrete
Lecteur 3 arrete
Lecteur 1 arrete

=== STATISTIQUES FINALES ===
Lectures: 6
Ecritures: 3
Valeur finale: Donnee-3-par-1

Contrôles
```

FIGURE 14 – Fin de simulation et statistiques finales

Explication :

- **>> ARRET DE LA SIMULATION <<** : Signal de fin envoyé par le simulateur.
- **Lecteur X arrete / Ecrivain X arrete** : Chaque thread confirme son arrêt.
- **=== STATISTIQUES FINALES ===** : Résumé de la simulation :
 - Nombre total de lectures effectuées
 - Nombre total d'écritures effectuées
 - Valeur finale de la ressource

5 Compilation et exécution

```
(ilyas@kali)-[~/lect-ecriv/src]
$ ls
Ecrivain.java  InterfaceGraphique.java  Lecteur.java  Main.java  makefile  RessourcePartagee.java  Simulateur.java

(ilyas@kali)-[~/lect-ecriv/src]
$ make run
mkdir -p bin
javac -d bin Main.java
java -cp bin Main

(ilyas@kali)-[~/lect-ecriv/src]
$
```

FIGURE 15 – Compilation et exécution avec Makefile

Explication des commandes :

- **make run** : Commande principale qui compile et exécute le programme.
- **mkdir -p bin** : Crée le dossier bin pour les fichiers compilés.
- **javac -d bin Main.java** : Compile la classe Main dans le dossier bin.
- **java -cp bin Main** : Exécute le programme.

6 Conclusion

6.1 Résumé du travail réalisé

Nous avons développé un simulateur complet du problème des lecteurs-écrivains répondant à toutes les exigences :

- **Modélisation** : Threads lecteurs et écrivains
- **Synchronisation** : ReentrantReadWriteLock avec mode FIFO
- **Journalisation** : Affichage complet des actions + fichier .log

- **Interface graphique** : Swing avec visualisation temps réel
- **Anti-famine** : Stratégie FIFO garantie

6.2 Difficultés rencontrées

1. **Synchronisation des compteurs** : Utilisation de `synchronized` pour protéger l'accès aux variables partagées.
2. **Arrêt propre des threads** : Mise en place du drapeau `fini` combiné avec `interrupt()` et `join()`.
3. **Affichage temps réel** : Utilisation de `SwingUtilities.invokeLater()` pour les mises à jour de l'interface.
4. **Journalisation dans un fichier** : Gestion des dates, horodatage, thread-safety avec `synchronized`.

6.3 Conclusion générale

Ce projet nous a permis de mettre en pratique les concepts fondamentaux de la programmation concurrente en Java :

- Gestion des threads et de leur synchronisation
- Utilisation des verrous lecture/écriture
- Développement d'une interface graphique temps réel
- Prévention de la famine avec une stratégie FIFO
- Journalisation des actions dans un fichier horodaté

Le simulateur fonctionne correctement et illustre parfaitement le problème des lecteurs-écrivains ainsi que sa solution.